

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1402	Course Title:	COMPILER DESIGN
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	40% of CIA		
CO1 Statement:	Apply lexical analysis for token specification and recognition using LEX tool. (BL3)		
Student Name:	AKSHAYA A	Reg. No.:	192421222
Section / Batch:	SLOT C	Date:	

Code of Conduct

I Akshaya A / 192421222 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Akshaya A

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	3 / 4	3 / 4	4 / 4	4 / 4	3 / 4	86.25
2	4 / 4	3 / 4	3 / 4	4 / 4	4 / 4	88.75
3	3 / 4	4 / 4	4 / 4	4 / 4	3 / 4	91.25
4	4 / 4	4 / 4	3 / 4	3 / 4	4 / 4	88.75
5	3 / 4	3 / 4	4 / 4	4 / 4	4 / 4	90
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	89.
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

N.D.A

CO1-AT1 - ANALYTICAL PROBLEM SOLVING

NAME: AKSHAYA. A

REG. NO: 192421222

COURSE: CSA1402 COMPILER DESIGN

COI-AT1-ANALYTICAL PROBLEM SOLVING

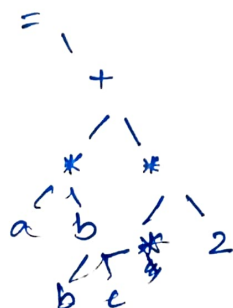
compiler phases for $P = (a * b) + (b * c) * 2$

a)

Lexeme	Token
P	Identifier
=	Assignment operator
(	Left Paranthesis
a	Identifier
*	Multiplication Operator
b	Identifier
)	Right Paranthesis
+	Addition Operator
(	Left Paranthesis
b	Identifier
*	Multiplication operator
c	Identifier
)	Right Paranthesis
*	Multiplication Operator
2	Integer constant

b)

syntax tree



c) Intermediate code
 $t_1 = a * b$ $t_2 = b * c$ $t_3 = t_2 * 2$
 $t_4 = t_1 + t_3$ $p = t_4$

d) Optimized code

$$t_1 = a * b$$

$$t_2 = b * c$$

$$P = t_1 + (t_2 * 2)$$

e) Significant of Target code

Machine dependent executable instructions

Optimized for efficient execution

Can be directly executed by processor

2)

a) $\epsilon (aa^*)^*$

strings: $\epsilon, a, aa, aaa, aaaa$

b) $(ab)^* abb$

strings: $abb, aabb, babb$

c) $(ab) aba (a+b^*)^*$

strings: $aba, abaa, baba$

d) $(aab)^*$

strings: $\epsilon, aab, aabaab, aabaabaab$

$$(aba)^* + (ab)^*$$

strings: ϵ , aba, abaaba

3)
i) No. of Tokens : 30

ii) Lexical Error: 1xab is invalid.
Correct: 0xAB.

4) Error: void main()
{
 int x = 10, y = 20;
 char * a;
 a = &x;
 x = 1xab;
}

4) Regular Expressions

i) Even number of 0s : $1^*(01^*01^*)^*$

ii) At least one 1 : $(011)^*1(011)^*$

iii) Both number of 0s & 1s are odd

$$(1^*01^*01^*)^*(1^*01^*)$$

$$(0^*10^*10^*)^*(0^*10^*)$$

iv) No consecutive 1s: $(0|10)^* 1?$

v) Starts with 1 & ends with 0
 $(0|1)^* 0$

5) a) No. of Tokens

```
int var123 = 12345;  
float var456 = 123.45;  
if (var123 == var456)  
{  
    return;  
}
```

$\therefore$ Total tokens = 18

b) Keywords: int, float, if

Operators: =, ==

Integer: 12345

Identifiers: var123

Separators: (,), {, }, ;

Float: 123.45

c)

Token type	Approx. Transitions
Keywords	18
Identifier	24
Operators	4
Separators	7
Integer Literal	5
Float Literal	6
	<u>64</u>

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/for mula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited unclear or interpretation	No interpretation